

Self-Disguise Attack: Induce the LLM to disguise itself for AIGT detection evasion

Yinghan Zhou¹, Juan Wen¹, Wanli Peng¹, Zhengxian Wu¹, Ziwei Zhang¹, Yiming Xue¹,

¹College of information electrical and engineering, China Agricultural University
{zhouyh, wenjuan, wlpeng, wzxian, zzwei, xueym}@cau.edu.cn

Abstract

AI-generated text (AIGT) detection evasion aims to reduce the detection probability of AIGT, helping to identify weaknesses in detectors and enhance their effectiveness and reliability in practical applications. Although existing evasion methods perform well, they suffer from high computational costs and text quality degradation. To address these challenges, we propose Self-Disguise Attack (SDA), a novel approach that enables Large Language Models (LLM) to actively disguise its output, reducing the likelihood of detection by classifiers. The SDA comprises two main components: the adversarial feature extractor and the retrieval-based context examples optimizer. The former generates disguise features that enable LLMs to understand how to produce more human-like text. The latter retrieves the most relevant examples from an external knowledge base as in-context examples, further enhancing the self-disguise ability of LLMs and mitigating the impact of the disguise process on the diversity of the generated text. The SDA directly employs prompts containing disguise features and optimized context examples to guide the LLM in generating detection-resistant text, thereby reducing resource consumption. Experimental results demonstrate that the SDA effectively reduces the average detection accuracy of various AIGT detectors across texts generated by three different LLMs, while maintaining the quality of AIGT.

code — <https://github.com/CAU-ISS-Lab/AIGT-Detection-Evade-Detection/tree/main/SDA>

Introduction

Large language models (LLMs) such as deepseek (DeepSeek-AI 2024), GPT-4 (OpenAI 2024) and Qwen (Yang et al. 2024) have made a profound impact on both industrial and academic fields. Despite their impressive performance, LLMs have also raised significant concerns regarding their potential misuse, such as fake news (Su, Cardie, and Nakov 2024; Hu et al. 2024), academic dishonesty (Wu et al. 2023; Zeng, Sha, and Li 2024) and deceptive comments designed to manipulate public perception (Miresghallah et al. 2024). In response to these emerging threats, there is an increasing emphasis on developing robust and reliable methods for detecting AI-generated texts (AIGT) (Song et al. 2025; Bao et al. 2025; Soto et al. 2024). As detection capabilities of current AIGT detection methods improve, researchers have also

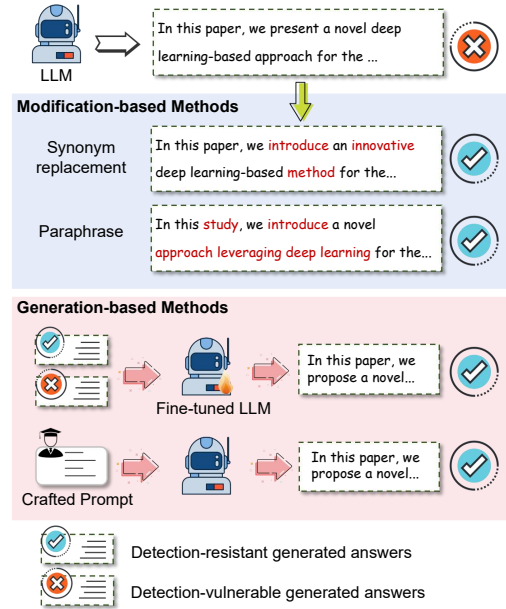


Figure 1: Overview of evasion detection paradigms. (1) The modification-based methods modify the generated text directly. (2) The generation-based approaches guide the LLM to generate detection-resistance text through fine-tuning and manually designing prompts.

begun exploring evasion techniques to better understand potential weaknesses in AI detectors and enhance their robustness before real-world deployment (Krishna et al. 2023; Zhou, He, and Sun 2024).

The evasion detection methods aim to decrease the detection probability of AIGT. As illustrated in Figure 1, they can be classified into two categories: the modification-based methods and the the generation-based methods. The first category modifies the generated text directly to evade detection, typically by leveraging common text attack techniques such as paraphrasing (Krishna et al. 2023) and synonym replacement (Zhou, He, and Sun 2024; Wang et al. 2024). While these methods can effectively bypass AIGT detectors, especially when the target detector is accessible, they often come at the cost of degraded text quality and increased compu-

tational overhead, as each text instance requires additional processing. To avoid such additional modifications, many researchers shifted their focus toward generation-based methods, aiming to design strategies that enable LLMs to directly generate detection-resistant text through techniques such as fine-tuning (Nicks et al. 2023; Wang et al. 2025) or manually designing prompts (Lu et al. 2023). Although the existing generation-based methods have demonstrated remarkable performance in evading detection, several challenges remain: (1) Using fine-tuning techniques results in increased resource consumption. (2) Using prompts to control the output of LLMs limits the diversity of the generated text. Therefore, there is an urgent need to develop more efficient and practical solutions to evade text detection.

To address the above challenges, we propose Self-Disguise Attack (SDA), a novel approach that enables the LLM to actively disguise its output, making it harder to be identified by detectors. SDA integrates two core components: an adversarial feature extractor and a retrieval-based context examples optimizer.

We first design an adversarial feature extractor to capture the disguise features that make AIGT more similar to human-written text. The adversarial feature extractor consisting of a feature generator, a text generator, and a proxy detector. Both the feature generator and the text generator are black-box LLMs accessible via API. Under the guidance of capturing disguise features through the adversarial process among the three modules, LLMs are able to understand how to generate more human-like text. These features, described in natural language, also reveal the current linguistic characteristic differences between HWT and AIGT, providing a new perspective for distinguishing between them. We empirically find that these disguise features generated by Qwen-max (Yang et al. 2024) also help other target LLMs produce more human-like text, highlighting an inherent connection between the outputs of different LLMs.

Inspired by retrieval-augmented generation (RAG) (Lewis et al. 2020), we design a retrieval-based context examples optimizer to enhance the diversity of generated text. It leverages similarity-based retrieval to select the top- k detection-resistant context examples most relevant to the users query from the external knowledge base to guide LLMs in generating the corresponding response. Through this adaptive context example selection strategy, we further enhance the ability of different LLMs to generate detection-resistant text, while preserving the quality of AIGT. We evaluate the performance of the proposed SDA against various AIGT detectors on three LLMs. Experimental results demonstrate that SDA surpasses four baseline methods, achieving the best evasion detection performance. Our main contributions can be summarized as follows:

- We propose Self-Disguise Attack (SDA) that effectively guides the LLM to disguise its output in order to decrease detection probability. Experimental results demonstrate that our proposed method reduces the average detection accuracy of texts generated by three target LLMs across four detectors, while maintaining the quality of AIGT.
- We design an adversarial feature extractor to capture the

disguise features, which can effectively guide the LLM to understand how to generate detection-resistant text.

- Inspired by RAG, we design a retrieval-based context examples optimizer. By constructing an external knowledge base containing detection-resistant texts, the optimizer improves the selection of context examples, further enhancing the self-disguise ability of LLMs and mitigating the impact on the diversity of the generated text.

Related Work

AIGT detection methods

Mitchell et al. (Mitchell et al. 2023) proposed a text perturbation method to measure the log probabilities difference between original and perturbed texts. Su et al. (Su et al. 2023) proposed a zero-shot method that measures the log-probability difference between original text and perturbed text using text perturbation techniques, significantly improving AIGT detection performance. These methods detected AIGT by leveraging various statistical differences between AIGT and human-written text (HWT). Additionally, Tian et al. (Tian et al. 2024) introduced a length-sensitive multiscale positive-unlabeled loss, which enhanced the detection performance for short texts while maintaining the detection efficacy for long texts. To enhance the generalization ability of model in known target domains, Verma et al. (Verma et al. 2024) proposed Ghostbuster, a method that processes documents through a series of weaker language models, conducted a structured search over possible combinations of their features, and then trained a classifier on the selected features to predict whether the documents were AI-generated. Guo et al. (Guo et al. 2024) proposed a multi-task auxiliary, multi-level contrastive learning framework, DeTeCtive, which detects generated text by learning different text styles. These methods involved training models on large labeled datasets to detect AIGT. Existing detectors were capable of achieving high detection performance, which placed higher demands on evasion detection methods.

Detection evasion methods

To reveal vulnerabilities in AI detectors before they are deployed in real-world applications, Krishna et al. (Krishna et al. 2023) conducted a paraphrasing attack method named discourse paraphraser (DIPPER). This method fine-tuned T5 (Raffel et al. 2020) by aligning, reordering, and calculating control codes, enabling the model to perform diverse modifications and paraphrasing of text without altering its original meaning. Zhou et al. (Zhou, He, and Sun 2024) proposed humanizing machine-generated content (HMGC). HMGC calculated the importance score for each token, and employed a masked language model to perform synonym replacement on the token with highest score until it can not be detected by the proxy detector. Wang et al. (Wang et al. 2025) proposed Humanized proxy attack (HUMPA), which fine-tunes a proxy small model using reinforcement learning (RL) and modifies the output probabilities of the target LLM based on the probability changes before and after fine-tuning the proxy model. To leverage the capabilities of LLMs, Lu et al. (Lu et al. 2023) proposed the substitution-based in-context

example optimization (SICO) method. This method guided the model to generate more human-like text by using samples that have undergone synonym replacement and paraphrasing as in-context examples. Although these methods achieved good performance in evading detection, they were unable to find a trade-off between resource consumption and the quality of the AIGT. Therefore, to reduce resource consumption while safeguarding text quality, we propose the SDA. Its details will be introduced in the following sections.

Method

Problem formulation

Given the user query set $\mathcal{X}$ and an input $x \in \mathcal{X}$, the text generated by LLM M based on x is defined as $t \sim P_M(\cdot | x)$. The detection probability of t by detector D is denoted as $p_D(t)$. The goal of the detection evasion task is to construct a function F that systematically adjusts t , ensuring that the detection probability $p_D(F(t))$ falls below a specified threshold σ . The task can be formulated as:

$$\begin{aligned} \tilde{t} &= \arg \min_{t'=F(t)} p_D(t') \\ \text{s.t. } p_D(\tilde{t}) &< \sigma, \mathcal{D}(t, \tilde{t}) \leq \epsilon, \end{aligned} \quad (1)$$

where the function $\mathcal{D}(t, \tilde{t})$ measures the distance between the original text t and transformed text $\tilde{t}$. The parameter ϵ controls the allowable modification extent to preserve the readability and semantics of texts.

In this work, our goal is to design a prompt z such that, for any given user input $x \in \mathcal{X}$, the text $t_z \sim P_M(\cdot | x \oplus z)$ remains undetectable, where $\oplus$ denotes the concatenation of the user input and the optimized prompt. Formally, our goal is defined as:

$$\begin{aligned} z^* &= \arg \min_z \mathbf{E}_{t_z \sim P(\cdot | x \oplus z)} [p_D(t_z)] \\ \text{s.t. } p_D(t_z) &< \sigma, \forall x \in \mathcal{X}. \end{aligned} \quad (2)$$

Self-Disguise Attack

In this work, we propose a novel method, Self-Disguise Attack (SDA), to address the challenges of evading AIGT detection effectively. The overall framework of SDA is shown in Figure 2. The SDA includes two main components: an adversarial feature generator and a retrieval-based context examples optimizer. The feature extractor captures key disguise features that enable LLMs to understand how to generate more human-like text. The retrieval-based optimizer further enhances evasion performance by searching an external knowledge base to identify the most relevant successfully disguised sentences to serve as context examples for the user query. Finally, we design a self-disguise generation prompt to guide the target LLM in generating detection-resistant text during the. As illustrated in Figure. 2, the final prompt consists of: (1) context examples selected by the optimizer based on user query, (2) extracted disguise features, and (3) a generation instruction.

Adversarial Feature Extractor

The adversarial feature extractor comprises a text generator, a feature generator, and a proxy detector.

The text generator is a black-box LLM accessed via API, responsible for generating text using the text generation prompt, which includes a generation instruction as mentioned above and the current disguise features as shown below: "*The features of texts resembling human writing are as follows: { }.*" The current disguise features are set to empty in the initial stage. In our experiments, we use Qwen (Yang et al. 2024) as the text generator to generate the candidate text.

The feature generator is a black-box LLM accessible via API, responsible for generating disguise features composed of various textual characteristics. To guide the generation of disguise feature, we design a *feature construction prompt*. As shown in Figure. 2, it consists of two main components: in-context examples and a detailed instruction. (1) *In-context examples*. We first utilize the proxy detector to evaluate the text generated by the text generator. Those identified as detection-resistant are then collected as contextual examples to construct feature construction prompts, guiding the feature generator in producing disguise features. (2) *Detailed instruction*. Previous studies (Doughman et al. 2025; Soto et al. 2024) have shown that AIGT and HWT exhibit differences in various textual characteristics. To bridge the gap, we explicitly specify the textual characteristics, including syntactic structure, grammatical structure, language style, punctuation usage and vocabulary choice, that the disguise features should focus on in the detailed instruction. Additionally, to eliminate the influence of text semantics, we require the feature generator to disregard the thematic content when generating features.

The proxy detector. To refine feature optimization, we employ an existing AIGT detector as a proxy detector to identify whether the generated text is AI-generated.

The complete process of the algorithm is shown in Appendix A. Specifically, we define an optimized step η and a maximum error range δ . For each query $x \in \mathcal{X}$, we first use the text generator to generate the corresponding response. The generated texts are then detected using a proxy detector. We collect texts that successfully bypass detection as context examples. Once η context examples are collected, they are provided to the feature generator for updating the current features. The text generator then continues generating text based on these refined features. The process above is repeated to extract the critical disguise features. Notably, if the number of AI-detected texts in two consecutive iterations does not exceed δ , the loop ends. Our approach does not require the incorporation of HWT, thereby effectively mitigating the risk of data contamination caused by the widespread use of LLMs during the collection of HWT. Additionally, this feature explains the linguistic differences between AIGT and HWT, providing a new interpretable perspective to distinguish between the two. The complete features we extracted and the analysis of the rationality of these features are shown in Appendix B.